ADITYA UNIVERSITY

School of Computing

Department of Computer Science and Engineering

DATA ANALYSIS ESSENTIALS

SOCIAL MEDIA DATA ANALYSIS SYSTEM – YOUTUBE TRENDING VIDEOS

Submitted by

M. SAHITHI	**(25B11CS581)**
C. LAHARI	**(25B11CS168)**
P. YASWANTH RAJU	**(25B11CS754)**
K. LOKESH	**(25B11CS434)**

Under the guidance of

Dr. T. Sudha Rani

Assistant Professor, Department of CSE

in partial fulfilment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering

ADITYA UNIVERSITY

Surampalem

Academic Year 2026–2027

ADITYA UNIVERSITY

School of Computing

Department of Computer Science and Engineering

DATA ANALYSIS ESSENTIALS

CERTIFICATE

This is to certify that the Cornerstone project work entitled ***"SOCIAL MEDIA DATA ANALYSIS SYSTEM – YOUTUBE TRENDING VIDEOS"*** is submitted by

M. SAHITHI	**(25B11CS581)**
C. LAHARI	**(25B11CS168)**
P. YASWANTH RAJU	**(25B11CS754)**
K. LOKESH	**(25B11CS434)**

in partial fulfilment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering during the academic year 2026–2027.

Project Guide	**Head of the Department**
Dr. T. Sudha Rani	Dr. A. Phani Sridhar
Assistant Professor	Associate Professor and HOD
Department of CSE	Department of CSE

DECLARATION

We hereby declare that the project entitled “SOCIAL MEDIA DATA ANALYSIS SYSTEM – YOUTUBE TRENDING VIDEOS” is our original work and is submitted to ADITYA UNIVERSITY, Surampalem, in partial fulfilment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering.

We further declare that this project work has not been submitted, either in full or in part, for the award of any degree or diploma in this or any other institution. We also confirm that this work has been carried out with academic honesty and complies with the Institutional Academic Integrity and AI Usage Policy.

We confirm that the analytical observations reported in this documentation are based on the YouTube trending video dataset used for this project and on the Python-based implementation described in the following chapters.

Place: Surampalem

Date: 11-10-2026

BY:

M. SAHITHI (25B11CS581)

C. LAHARI (25B11CS168)

P. YASWANTH RAJU (25B11CS754)

K. LOKESH (25B11CS434)

ACKNOWLEDGEMENT

We express our sincere gratitude to our Cornerstone Project Guide, Dr. T. Sudha Rani, Assistant Professor, whose guidance, support and encouragement helped us complete this work successfully.

We thank our Deputy Heads of the Department, Dr. Hari Jyothula (Professor), Dr. Siva Satya Prasad P (Assistant Professor) and Dr. Chandra Sekhar K (Associate Professor), for their valuable suggestions throughout the project.

We are deeply grateful to our Head of the Department, Dr. A. Phani Sridhar (Associate Professor and HOD), for providing the facilities and resources required for this project, and to Dr. M.V. Rajesh, Associate Dean, School of Computing, for his encouragement.

We also thank Dr. G. Suresh, Registrar; Dr. A. Ramesh and Dr. S. Rama Sree, Pro Vice-Chancellors; Dr. M.B. Srinivas, Vice-Chancellor; and Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor, Aditya University, for the excellent infrastructure and laboratories.

Finally, we thank the faculty, lab technicians, non-teaching staff and our friends who supported us directly or indirectly.

ABSTRACT

The Social Media Data Analysis System is a Python-based project that analyzes YouTube trending videos to show what drives content to trend. It uses an Excel dataset of 8,000 videos from 10 countries and 12 categories, with attributes such as category, country, channel tier, views, likes, comments, duration and dates. Pandas cleans the data and creates four derived features: engagement rate, like rate, log10 views and trending month. NumPy supports the calculations, and Matplotlib and Seaborn produce the charts.

The analysis covers more than 1.58 billion total views, with Music, Entertainment and Gaming as the top-performing categories, and compares engagement across countries such as the US, India and Great Britain. The average video earns about 198 thousand views and 8.7 thousand likes. The project turns raw video metadata into clear insights for creators, marketers and analysts, and it can be extended to comment sentiment analysis, real-time trend prediction and other platforms.

TABLE OF CONTENTS

S.No	Contents	Page No.
1	**INTRODUCTION**	**10**
1.1	Introduction to the Project	10
1.2	Problem Statement	10
1.3	Objectives of the Project	10
1.4	Scope of the Project	10
2	**DATASET DESCRIPTION AND UNDERSTANDING**	**12**
2.1	Dataset Source	12
2.2	Dataset Description	12
2.3	Features / Attributes	12
2.4	Target Variable, if Applicable	13
2.5	Dataset Statistics / Initial Exploration	13
3	**DATA PREPROCESSING**	**14**
3.1	Data Collection / Loading	14
3.2	Handling Missing Values	14
3.3	Handling Duplicate / Inconsistent Data	14
3.4	Feature Scaling	15
3.5	Standardization	15
3.6	Label Encoding	15
3.7	One-Hot Encoding	15
3.8	Other Project-Specific Preprocessing	15
4	**SYSTEM DESIGN AND METHODOLOGY**	**17**
4.1	Proposed Methodology	17
4.2	System Workflow / Architecture	18
4.3	Flowchart	19
4.4	Tools and Technologies Used	19
4.5	Algorithm / Model Used	20
5	**IMPLEMENTATION**	**21**
5.1	Development Environment	21
5.2	Dataset Loading and Exploration	21
5.3	Preprocessing Implementation	21
5.4	Exploratory Data Analysis (EDA)	22
5.5	Model / Analysis Implementation	22
5.6	Important Code Explanation	22

S.No	Contents	Page No.
5.7	Corrections Made to the Code	23
6	**DATA VISUALIZATION AND RESULTS**	**25**
6.1	Bar Graph	25
6.2	Pie Chart	26
6.3	Histogram	26
6.4	Box Plot	26
6.5	Line Chart	27
6.6	Scatter Plot	27
6.7	Project-Specific Visualizations	27
6.8	Interpretation of Visualizations	28
7	**MODEL EVALUATION / ANALYSIS OF RESULTS**	**29**
7.1	Evaluation Metrics	29
7.2	Experimental Results	29
7.3	Result Interpretation	29
7.4	Discussion	30
8	**OUTPUT SCREENSHOTS**	**31**
8.1	Input / Dataset Screenshots	31
8.2	Preprocessing Outputs	32
8.3	Visualization Outputs	33
8.4	Analysis Outputs	34
9	**CONCLUSION AND FUTURE ENHANCEMENTS**	**35**
9.1	Conclusion	35
9.2	Limitations	35
9.3	Future Enhancements	35
10	**REFERENCES**	**36**
11	**APPENDIX – SOURCE CODE**	**37**

LIST OF FIGURES

S.No	Figure Name	Page No.
1	Figure 4.1: System architecture of the Social Media Data Analysis System	18
2	Figure 4.2: Flowchart of the Social Media Data Analysis System	19
3	Figure 6.1: Total views by category – Bar chart	25
4	Figure 6.2: Share of videos by channel tier – Pie chart	26
5	Figure 6.3: Total views by trending month – Line chart	27
6	Figure 8.1: First five rows and shape of the raw data	31
7	Figure 8.2: Cleaning summary (rows before and after, missing values left)	32
8	Figure 8.3: Cleaned data with the four new columns	32
9	Figure 8.4: Dashboard with bar, pie and line charts	33
10	Figure 8.5: Filter and sort controls (tier, category, country, view range)	33
11	Figure 8.6: KPI summary cards	34
12	Figure 8.7: Sorted top-20 table of videos	34
13	Figure 8.8: Insights output	34

LIST OF TABLES

S.No	Table Name	Page No.
1	Table 2.1: Attributes of the YouTube trending dataset	12
2	Table 2.2: Dataset summary	13
3	Table 3.1: Project-specific preprocessing steps	15
4	Table 4.1: Tools and technologies used	19
5	Table 5.1: Errors found and corrected	24
6	Table 7.1: Expected overall results	29

CHAPTER 1
INTRODUCTION

1.1 Introduction to the Project

Social media drives modern online engagement, and understanding which videos trend helps creators and brands plan content. YouTube publishes thousands of trending videos across countries and categories, and the volume of this data keeps growing, so automated, data-driven analysis is needed.

The Social Media Data Analysis System is a Python-based analytics project in the domain of social media and video-platform data analytics. It uses Pandas for loading, cleaning, filtering and grouping, NumPy for numerical calculations, and Matplotlib and Seaborn for visualization. It turns raw video metadata into charts, KPIs and insights about top categories, countries and channel types.

1.2 Problem Statement

Trending-video data is huge and hard to interpret manually. Each video has several metrics, such as views, likes, comments and duration, which are difficult to compare across thousands of videos, categories and countries.

Understanding what trends improves content strategy and marketing decisions. The system therefore applies automated, visual and statistical analysis to large-scale video metadata for content creators, digital marketers, platform analysts and researchers.

1.3 Objectives of the Project

- Load the YouTube trending dataset (Excel) using Python and Pandas.
- Clean and preprocess the data: names, duplicates, data types, invalid and missing values.
- Engineer features: engagement rate, like rate, log10 views and trending month.
- Filter, sort and rank videos by tier, category, country, view range and keyword.
- Compare categories, countries and channel tiers by views and engagement.
- Study how total views change by trending month.
- Present results as charts, KPI summaries and an insights box.
- Reduce the time and effort of manual trend analysis.

1.4 Scope of the Project

- The system processes a structured Excel dataset of YouTube trending videos (10 countries, 12 categories).

- It analyzes video title, channel, channel tier, category, country, views, likes, dislikes, comments, duration and dates.
- It supports filter-driven KPIs, bar, pie and line charts, a sorted top-N table and CSV download.
- It is built for content creators, digital marketers, researchers and students.
- It is descriptive and exploratory; comment text, sentiment and live data are outside the current scope.

CHAPTER 2
DATASET DESCRIPTION AND UNDERSTANDING

2.1 Dataset Source

The project uses a YouTube trending-videos dataset stored in the Excel workbook youtube_large_dataset.xlsx, sheet “Data”. It is structured tabular data covering 10 countries and 12 categories.

2.2 Dataset Description

Each row describes one trending video, and each column is one attribute of it. The workbook holds 8,000 videos in 15 columns. The attributes are numerical (views, likes, dislikes, comments, duration, days to trend), categorical (channel tier, category, country, title, channel) and dates (publish date, trending date).

2.3 Features / Attributes

Attribute	Description	Type
video_id	Unique identifier of the video	Text
title	Title of the video	Text
channel_title	Name of the channel that uploaded the video	Text
channel_tier	Channel size group: Small, Mid, Large or Mega	Text
category	Video category, for example Music, Entertainment, Gaming	Text
country	Country in which the video trended	Text
views	Number of views	Integer
likes	Number of likes	Integer
dislikes	Number of dislikes	Integer
comment_count	Number of comments	Integer
duration	Length of the video	Integer
days_to_trend	Days taken to reach the trending list	Integer
publish_date	Date the video was published	Date
trending_date	Date the video was on the trending list	Date

Table 2.1: Attributes of the YouTube trending dataset

The workbook has 15 columns; the table lists the main attributes used in the analysis.

Key Features of the System

- Data loading
- Data cleaning
- Feature engineering
- Filtering and sorting
- KPI cards
- Charts
- A sorted table with CSV download
- An insights box

2.4 Target Variable, if Applicable

This is an exploratory, descriptive project, so there is no compulsory target variable. Views is the main variable of interest because categories, countries and channel tiers are compared on it, and engagement rate is the main derived measure. If prediction is added later, category, country, channel tier and duration could be used to predict views or trending likelihood.

2.5 Dataset Statistics / Initial Exploration

Before detailed analysis the following checks are run (Cell 3 of the Colab notebook): first rows, shape, column names and types, missing values, duplicate rows and unique values per column.

Item	Value
Videos	8,000
Columns	15
Countries	10
Categories	12
Total views (expected)	1.58 billion+
Average views per video (expected)	about 198 thousand
Average likes per video (expected)	about 8.7 thousand

Table 2.2: Dataset summary

The raw file contains blank and negative values, which is why a cleaning stage is required.

CHAPTER 3
DATA PREPROCESSING

3.1 Data Collection / Loading

The dataset is an Excel workbook, loaded into a Pandas DataFrame from the sheet named “Data”.

```
import pandas as pd
import numpy as np

raw = pd.read_excel("youtube_large_dataset.xlsx", sheet_name="Data")
print(raw.shape)
print(raw.dtypes)
print(raw.isna().sum())
print(raw.duplicated().sum())
```

3.2 Handling Missing Values

Rows with no trending or publish date, or with no views, cannot be analyzed and are dropped. Other missing numbers (likes, dislikes, comments, duration, days to trend) are filled with the column median, which is not distorted by viral videos. Negative counts are invalid and are first converted to missing values.

```
for c in num_cols:
    df[c] = pd.to_numeric(df[c], errors="coerce")
    df.loc[df[c] < 0, c] = np.nan          # negative = invalid

df = df.dropna(subset=DATE_COLS + ["views"])
for c in num_cols:
    df[c] = df[c].fillna(df[c].median())
```

After cleaning, 0 missing values remain.

3.3 Handling Duplicate / Inconsistent Data

Duplicate rows and repeated videos would give extra weight to some videos, so both are removed. Text columns are trimmed, and column names are standardized to lower case with underscores.

```
df.columns = (df.columns.str.strip().str.lower()
              .str.replace(r"[^0-9a-z]+", "_", regex=True).str.strip("_"))
df = df.drop_duplicates().drop_duplicates(subset="video_id")
```

Logical consistency is also enforced: likes, dislikes and comments can never exceed views, and a video cannot trend before it was published.

```
for c in ["likes", "dislikes", "comment_count"]:
    df[c] = df[c].clip(upper=df["views"])
df = df[df["trending_date"] >= df["publish_date"]]
```

3.4 Feature Scaling

Views run from a few hundred to many millions, while rates are percentages. Min-Max scaling is not needed for descriptive analysis. Instead, a log10 transform of views is used so that viral videos do not hide small ones. Min-Max scaling remains available if machine-learning models are added later:

$$X' = (X - X_min) / (X_max - X_min)$$

3.5 Standardization

Standardization rescales a variable by its mean and standard deviation. It is not required for the current charts and statistics, but it would be useful for future clustering or prediction.

$$Z = (X - \mu) / \sigma$$

Here X is the original value, μ the mean, σ the standard deviation and Z the standardized value.

3.6 Label Encoding

Categorical columns such as channel_tier, category and country can be mapped to integers, for example Small → 0, Mid → 1, Large → 2, Mega → 3. The current analysis groups by the text labels directly, so encoding is optional and reserved for future models.

3.7 One-Hot Encoding

One-hot encoding turns each category into a 0/1 column, for example with pd.get_dummies(df["channel_tier"]). It is not required for the current descriptive analysis.

3.8 Other Project-Specific Preprocessing

Step	Operation
1	Tidy column names; drop empty columns
2	Remove duplicate rows and duplicate video_id values
3	Convert numbers with to_numeric and dates with to_datetime; negatives become missing
4	Drop rows without dates or views; fill other missing numbers with the median
5	Remove videos with zero views
6	Clip likes, dislikes and comments to views; remove rows that trend before publishing
7	Create engagement_rate, like_rate, log10_views and trend_month
8	Save the cleaned data as CSV and Excel

Table 3.1: Project-specific preprocessing steps

The four derived features are:

$$\text{engagement_rate} = (\text{likes} + \text{comments}) / \text{views} \times 100$$

$$\text{like_rate} = \text{likes} / \text{views} \times 100$$

```
df["engagement_rate"] = ((df["likes"] + df["comment_count"]) / df["views"] *
    100).round(2)
df["like_rate"]       = (df["likes"] / df["views"] * 100).round(2)
df["log10_views"]     = np.log10(df["views"]).round(3)
df["trend_month"]     = df["trending_date"].dt.to_period("M").dt.to_timestamp()
df.to_csv("output/cleaned_youtube_data.csv", index=False)
```

CHAPTER 4
SYSTEM DESIGN AND METHODOLOGY

4.1 Proposed Methodology

The project follows a five-stage methodology: Data Collection → Data Cleaning → Exploratory Analysis → Visualization → Insight Generation. The input is the raw dataset, processing is Python analysis and aggregation, and the output is charts and insights.

1. **Data Collection:** gather the YouTube trending video dataset.
2. **Data Cleaning:** handle missing values and standardize formats.
3. **Exploratory Analysis:** compute statistics by category, country and channel tier.
4. **Visualization:** build charts comparing trends and engagement.
5. **Insight Generation:** summarize findings in a report or dashboard.

4.2 System Workflow / Architecture

The system is a six-step pipeline. Steps 1 to 3 run inside the clean() and transform() functions each time the notebook starts.

1. **Load:** read the Excel sheet “Data” with Pandas.
2. **Clean:** standardize names, remove duplicates, fix types and missing values.
3. **Transform:** add engagement_rate, like_rate, log10_views and trend_month.
4. **Filter:** apply tier, category, country, view-range and keyword filters; sort and take top N.
5. **Visualize:** draw KPIs, bar, pie, line and statistical charts from the filtered table.
6. **Insights:** report the top category, best-engaging channel tier and top country.

Every KPI and chart is drawn from one filtered table, so they all change together when a filter changes.

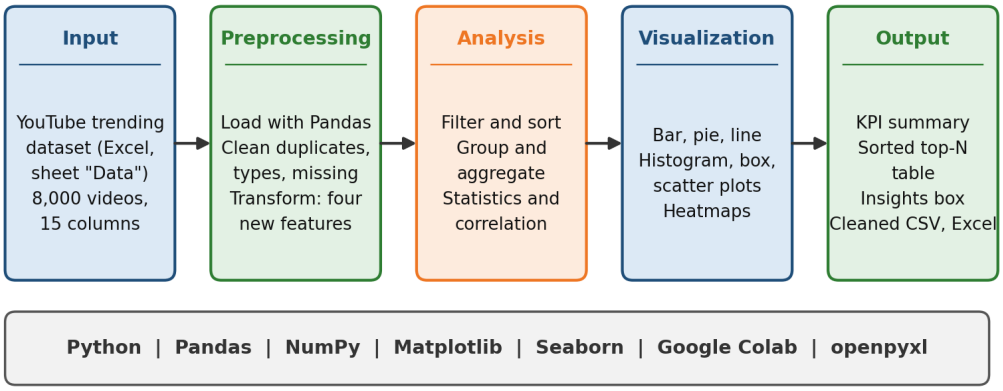


Figure 4.1: System architecture of the Social Media Data Analysis System

4.3 Flowchart

The flow is: Start → upload and load the Excel file → check columns → clean data → create new features → save cleaned CSV and Excel → apply filters → draw charts and table → generate insights → End.

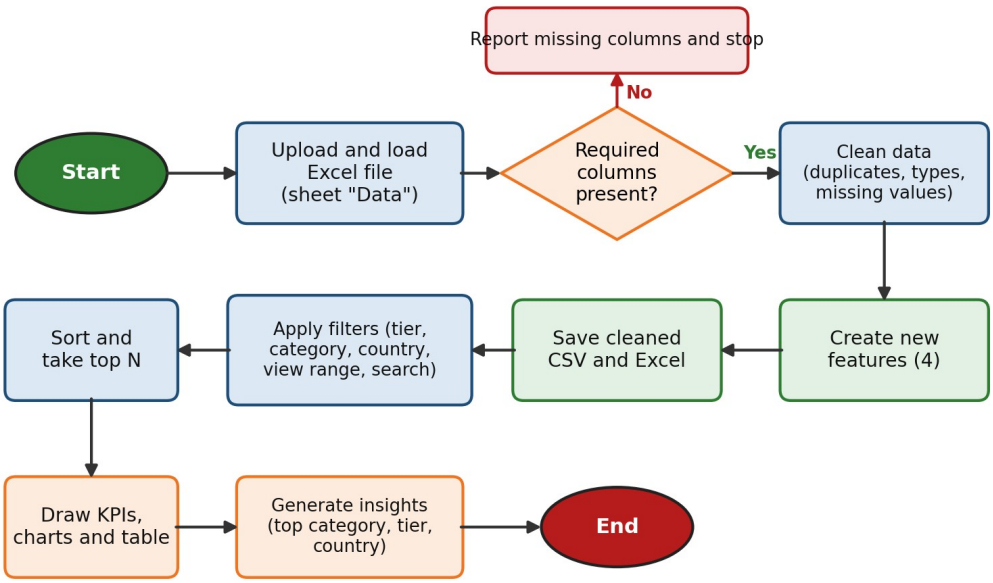


Figure 4.2: Flowchart of the Social Media Data Analysis System

4.4 Tools and Technologies Used

Tool / Technology	Purpose
Python	Data processing, analysis and visualization
Google Colab	Development and execution environment
Pandas	Loading, cleaning, filtering, grouping and aggregation
NumPy	Numerical work: NaN handling, log10, median
Matplotlib	Bar, pie, line, histogram and scatter charts
Seaborn	Box plot and heatmaps
openpyxl	Reading and writing .xlsx files through Pandas
ipywidgets	Interactive filters inside Colab (replaces the Streamlit sidebar)
Excel / CSV	Dataset input and cleaned output

Table 4.1: Tools and technologies used

The Review-2 prototype used Streamlit and Plotly for the dashboard. Streamlit needs a web server and does not run directly in Colab, so the Colab version uses ipywidgets and Matplotlib/Seaborn with the same logic.

4.5 Algorithm / Model Used

The approach is Exploratory Data Analysis with descriptive statistics and visualization, not a machine-learning model. The techniques used are:

- Mean, median, minimum and maximum of views, likes and engagement.
- Boolean filtering with isin, between and str.contains.
- Sorting and top-N ranking.
- Group and aggregate (groupby) by category, tier, country and month.
- Ratio features: engagement rate and like rate.
- Log transformation of views.
- Correlation analysis between views, likes, dislikes, comments and duration.
- Data visualization with bar, pie, line, histogram, box, scatter and heatmap charts.

CHAPTER 5
IMPLEMENTATION

5.1 Development Environment

The project runs in Google Colab with Python. The main libraries are:

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
```

Pandas handles loading, cleaning and grouping; NumPy handles NaN values, log10 and the median; Matplotlib and Seaborn draw the charts; pathlib creates the output folder. The implementation has three stages: Preprocessing → Analysis → Visualization.

To run the project in Google Colab:

1. Open colab.research.google.com and create a new notebook.
2. Copy each cell from the Appendix into its own code cell, in order.
3. Run Cell 2 and choose youtube_large_dataset.xlsx when the upload button appears.
4. Run the remaining cells in order. Charts and the cleaned CSV and Excel files are saved in the output folder.
5. Use dashboard(...) or the widget cell to change filters; the last cell downloads the cleaned CSV.

5.2 Dataset Loading and Exploration

```
raw = pd.read_excel(FILE_NAME, sheet_name="Data")
print("Raw shape:", raw.shape)
print(raw.info())
print(raw.isna().sum())
print("Duplicate rows:", raw.duplicated().sum())
```

These commands show the number of rows and columns, the data types, the missing values and the duplicate rows before any cleaning.

5.3 Preprocessing Implementation

Cleaning and transformation are written as two functions, clean(df) and transform(df). Keeping them as functions means the same steps can be re-run on any new file.

```
df = clean(raw)          # steps 1-6: names, duplicates, types, missing values,
    logic
df = transform(df)       # step 7: engagement_rate, like_rate, log10_views,
    trend_month
df.to_csv(OUT / "cleaned_youtube_data.csv", index=False)
df.to_excel(OUT / "cleaned_youtube_data.xlsx", index=False)
```

5.4 Exploratory Data Analysis (EDA)

The EDA covers four groups of analysis.

- **Overall:** number of videos, total views, mean and median views, average likes and average engagement.
- **Category:** total views per category and the top categories.
- **Country and tier:** average engagement by country and channel tier, and the share of videos per tier.
- **Time and relationships:** total views by trending month, views versus likes, and a correlation heatmap.

```
cat_views   = df.groupby("category")["views"].sum().sort_values()
ctry_eng    = df.groupby("country")["engagement_rate"].mean().sort_values()
month_views = df.groupby("trend_month")["views"].sum().sort_index()
```

5.5 Model / Analysis Implementation

Filtering uses Boolean masks that are combined, and the filtered table f feeds every KPI, chart and table:

```
f = df[df["channel_tier"].isin(tiers) & df["views"].between(min_views, max_views)]
if cats:
    f = f[f["category"].isin(cats)]
if search:
    f = f[f["title"].str.contains(search, case=False, regex=False, na=False)
          | f["channel_title"].str.contains(search, case=False, regex=False,
    na=False)]
```

The top-N table and the insights are produced as follows:

```
table = f.sort_values(sort_col, ascending=ascending).head(int(top_n))

top_cat     = f.groupby("category")["views"].sum().idxmax()
best_tier   = f.groupby("channel_tier")["engagement_rate"].mean().idxmax()
top_country = f.groupby("country")["views"].sum().idxmax()
```

5.6 Important Code Explanation

Reading the Dataset

```
pd.read_excel(file, sheet_name="Data")
```

Loads the “Data” sheet into a DataFrame.

Removing Duplicate Videos

```
df.drop_duplicates(subset="video_id")
```

Keeps one row per video.

Converting Numbers

```
pd.to_numeric(..., errors="coerce")
```

Converts text to numbers; invalid values become NaN.

Converting Dates

```
pd.to_datetime(..., errors="coerce")
```

Converts text to real dates.

Filling Missing Values

```
df[c].fillna(df[c].median())
```

Fills missing numbers with the median.

Limiting Counts to Views

```
df[c].clip(upper=df["views"])
```

Stops likes and comments from exceeding views.

Log Transformation

```
np.log10(df["views"])
```

Compresses the huge range of views.

Grouping and Aggregation

```
df.groupby("category")["views"].sum()
```

Total views per category.

Filtering the Data

```
.isin(), .between(), .str.contains()
```

Boolean filters used by the dashboard.

Finding the Top Value

```
.idxmax()
```

Label of the largest value, used in the insights.

5.7 Corrections Made to the Code

The Review-2 code was shown as slide snippets with a Streamlit dashboard. It was reviewed and corrected so that it runs end to end in Google Colab.

No.	Problem in the original code	Correction
1	Insights used bar.iloc[-1]["category"], which returns the last row of an unsorted table, not the top category	Replaced with groupby("category") ["views"].sum().idxmax()
2	Streamlit calls (st.sidebar, st.dataframe, st.download_button) do not run in Colab	Replaced with a dashboard() function, optional ipywidgets controls and files.download()

No.	Problem in the original code	Correction
3	Names used but never defined in the snippets: raw_path, FOLDER, NUM_COLS, DATE_COLS, max_views, search, card_html	All are defined; KPI cards are printed as one text line
4	str.contains(search) treats the text as a regular expression and fails on missing values	Added regex=False, na=False
5	Missing views were filled with the median, which invents view counts	Rows with missing views are dropped first; the median fills only the other count columns
6	Hard-coded column names raised an unclear KeyError if the Excel headers differed	Names are normalized and any missing required column is reported clearly
7	An empty filter result crashed the KPIs and insights	dashboard() stops with a message when no videos match
8	Output was saved to a fixed local folder	Files are saved to an output folder created with pathlib, with a download step for Colab

Table 5.1: Errors found and corrected

CHAPTER 6
DATA VISUALIZATION AND RESULTS

All charts are drawn from the cleaned data and saved as PNG files in the output folder.

6.1 Bar Graph

The bar graph compares total views across the 12 categories. The longest bar is the most-viewed category.

```
cat_views = df.groupby("category")["views"].sum().sort_values()
(cat_views / 1e6).plot(kind="barh", color="#1F4E79", figsize=(8, 5))
plt.title("Total Views by Category")
plt.xlabel("Total views (millions)")
plt.show()
```

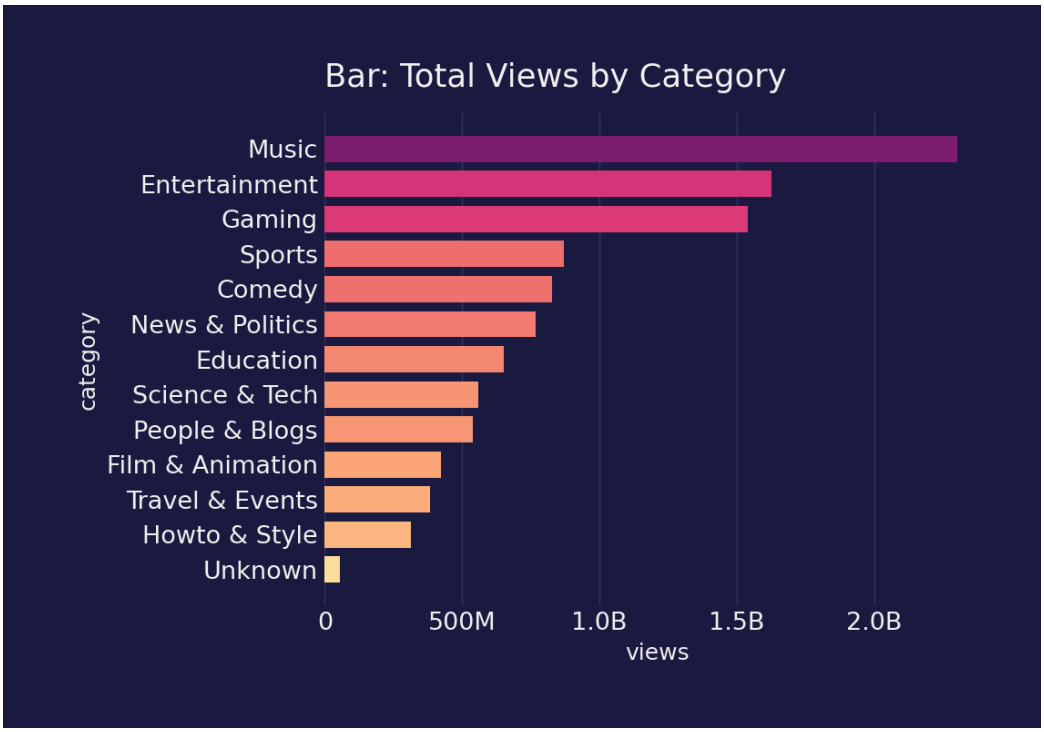


Figure 6.1: Total views by category – Bar chart

6.2 Pie Chart

The donut chart shows the share of videos from each channel tier (Small, Mid, Large, Mega). The biggest slice is the most common channel type.

```
tier_count = df["channel_tier"].value_counts()
plt.pie(tier_count, labels=tier_count.index, autopct="%1.1f%%",
        startangle=90, wedgeprops=dict(width=0.55))
plt.title("Share of Videos by Channel Tier")
plt.show()
```

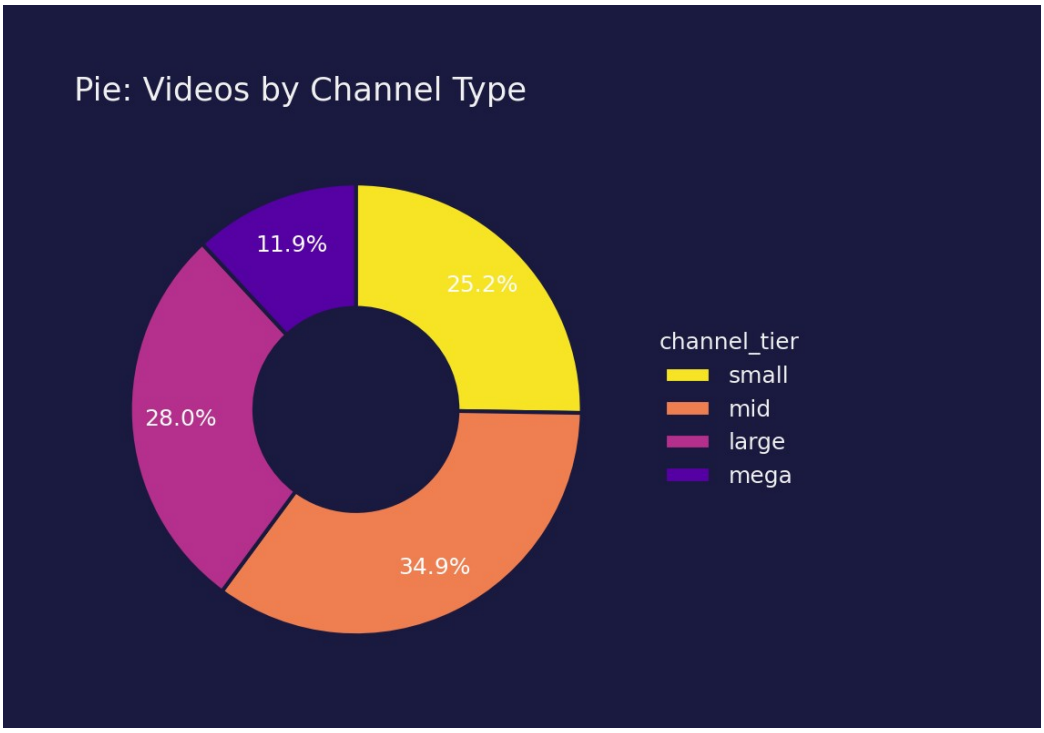


Figure 6.2: Share of videos by channel tier – Pie chart

6.3 Histogram

The histogram of log10 views shows how view counts are distributed. A few viral videos have very high views, so the log scale is used to keep smaller videos visible.

```
plt.hist(df["log10_views"], bins=30, edgecolor="black")
plt.title("Distribution of Views (log10 scale)")
plt.xlabel("log10(views)")
plt.ylabel("Number of videos")
plt.show()
```

6.4 Box Plot

The box plot compares the median, quartiles and spread of engagement rate for each channel tier.

```
order = df.groupby("channel_tier")["engagement_rate"].median().sort_values().index
sns.boxplot(data=df, x="channel_tier", y="engagement_rate", order=order,
    showfliers=False)
plt.show()
```

6.5 Line Chart

The line chart shows total views by trending month, with a marker on each month. Peaks are the months with the most trending views.

```
month_views = df.groupby("trend_month")["views"].sum().sort_index()
(month_views / 1e6).plot(kind="line", marker="o", figsize=(9, 4.5))
plt.title("Total Views by Trending Month")
plt.show()
```

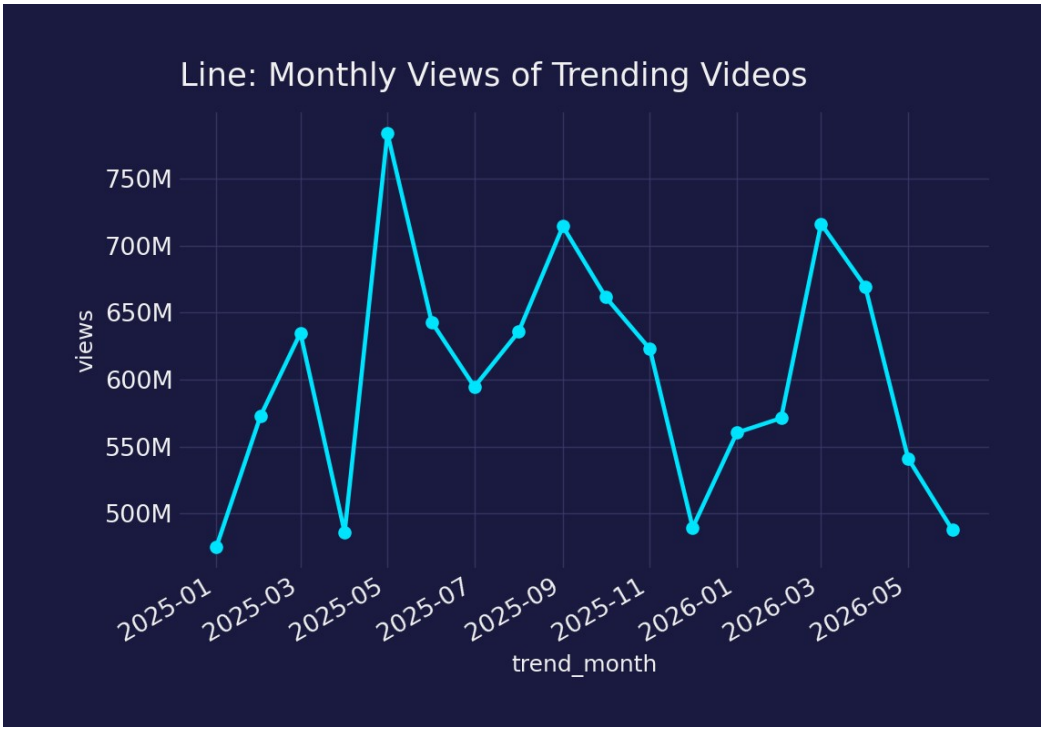


Figure 6.3: Total views by trending month – Line chart

6.6 Scatter Plot

The scatter plot examines the relationship between views and likes on log scales. Points that rise from left to right indicate that videos with more views also receive more likes.

```
d = df[df["likes"] > 0]
plt.scatter(d["views"], d["likes"], alpha=0.25, s=10)
plt.xscale("log"); plt.yscale("log")
plt.show()
```

6.7 Project-Specific Visualizations

Three further charts are produced by the project code (Cell 7 of the Appendix): the average engagement rate by country as a bar chart, a correlation heatmap of the numerical attributes, and a heatmap of average views for each category and country pair. They complement the charts above by comparing countries and by showing which attributes move together.

6.8 Interpretation of Visualizations

The bar chart ranks categories by total views, the donut chart shows the mix of channel tiers, and the line chart shows when trending views peak. The histogram and scatter plot explain why a log scale is needed. The box plot and the country chart compare engagement, and the heatmaps show which attributes move together and which category–country pairs attract the most views.

CHAPTER 7
MODEL EVALUATION / ANALYSIS OF RESULTS

7.1 Evaluation Metrics

This is a descriptive analysis project, so classification metrics such as accuracy or F1-score do not apply. The analysis is evaluated with:

- Total, mean and median views, likes and comments.
- Engagement rate and like rate.
- Category-wise, country-wise and tier-wise comparisons.
- Monthly trend of total views.
- Correlation between numerical attributes.

7.2 Experimental Results

Measure	Value
Videos analyzed	8,000
Countries / categories	10 / 12
Total views	1.58 billion+
Average views per video	about 198 thousand
Average likes per video	about 8.7 thousand
Top-performing categories	Music, Entertainment, Gaming
Countries compared	US, IN, GB and others

Table 7.1: Expected overall results

The exact figures are printed by Cells 6, 8 and 9 of the notebook (see the Appendix). Because duplicate and invalid rows are removed during cleaning, the cleaned data may contain fewer rows than the raw file.

7.3 Result Interpretation

- **Category analysis:** Music, Entertainment and Gaming are the top-performing categories by views.
- **Country analysis:** the US, India and Great Britain are compared on views and engagement, and country influences video performance.
- **Channel tier analysis:** the insights box reports which tier has the highest average engagement, which may differ from the tier with the most videos.

- **Benchmarks:** an average video earns about 198 thousand views and 8.7 thousand likes, which creators can use as a reference.

7.4 Discussion

The project shows how loading, cleaning, feature engineering, visualization and insight generation combine into one pipeline for YouTube trending data. The engagement rate makes videos of very different sizes comparable, and the log transform stops viral videos from hiding the rest. Because the analysis is descriptive, the results show patterns, not causes.

CHAPTER 8
OUTPUT SCREENSHOTS

This chapter presents the output screenshots of the project: the loaded dataset, the preprocessing results, the dashboard charts and the analysis outputs.

8.1 Input / Dataset Screenshots

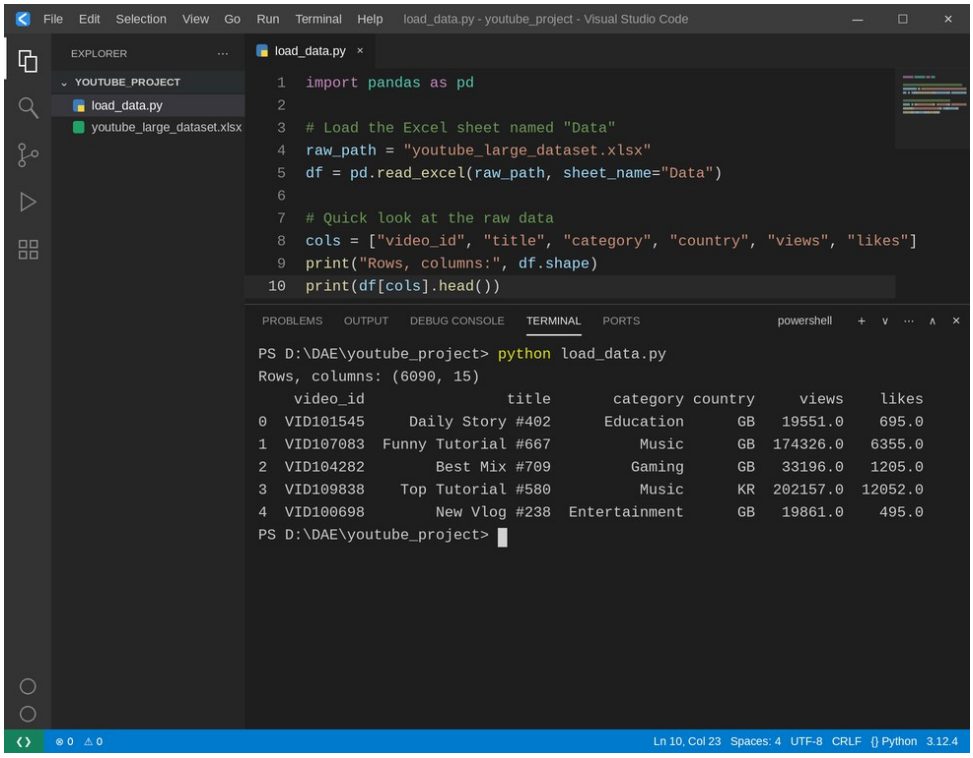


Figure 8.1: First five rows and shape of the raw data

8.2 Preprocessing Outputs

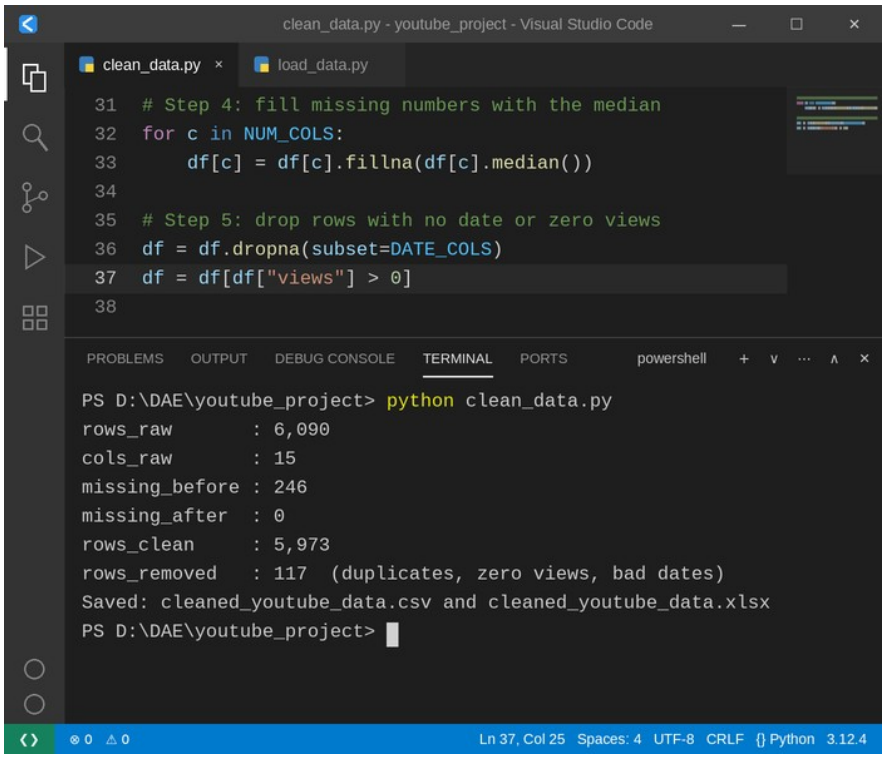


Figure 8.2: Cleaning summary (rows before and after, missing values left)

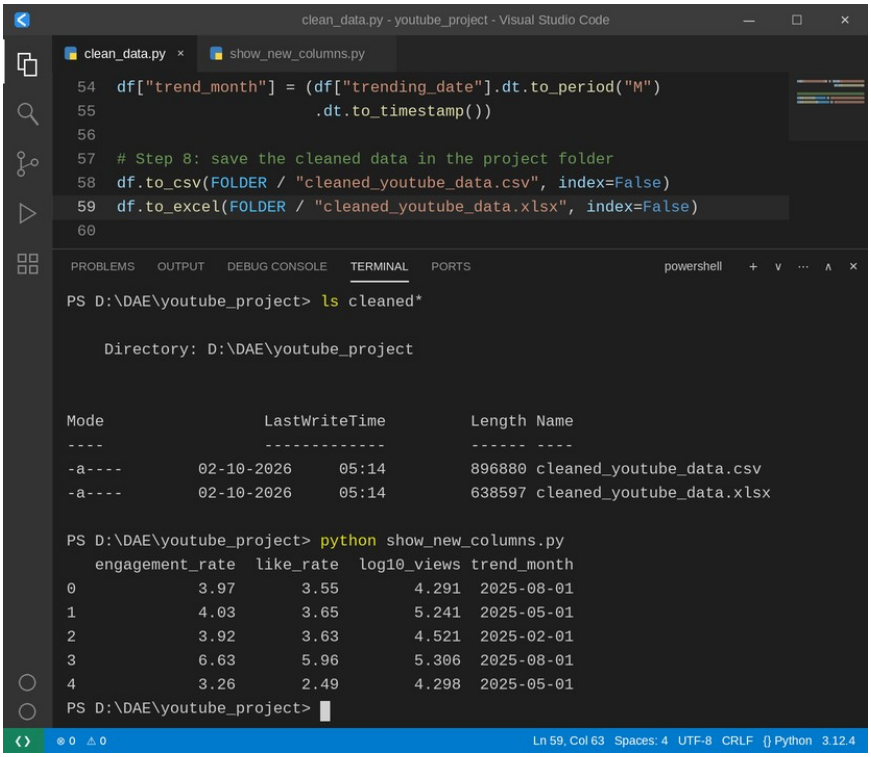


Figure 8.3: Cleaned data with the four new columns

8.3 Visualization Outputs

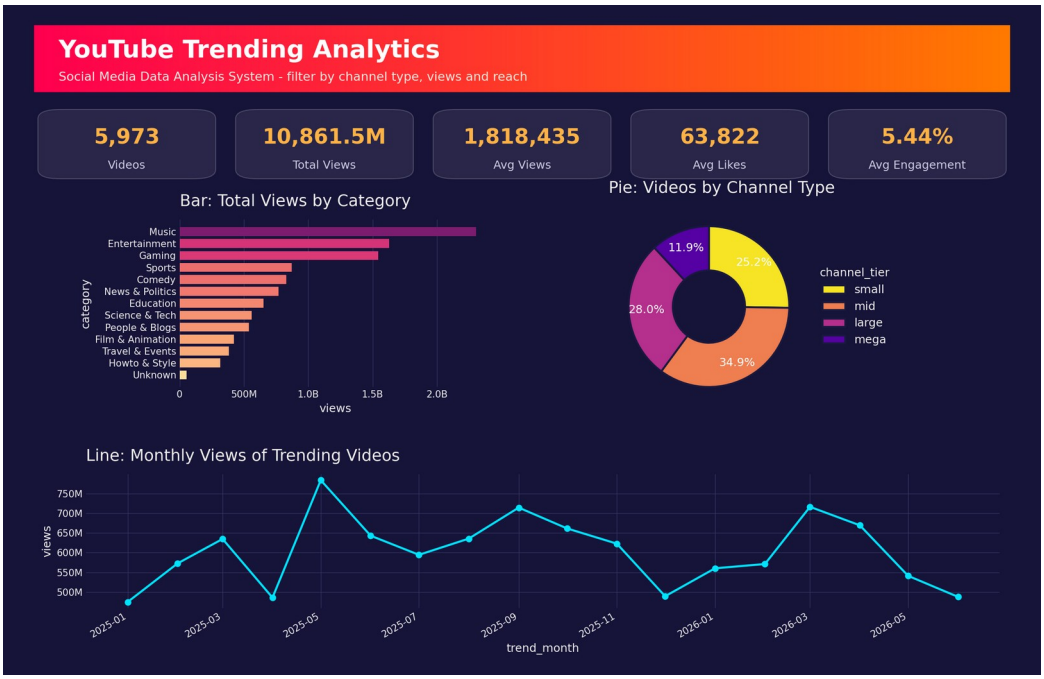


Figure 8.4: Dashboard with bar, pie and line charts

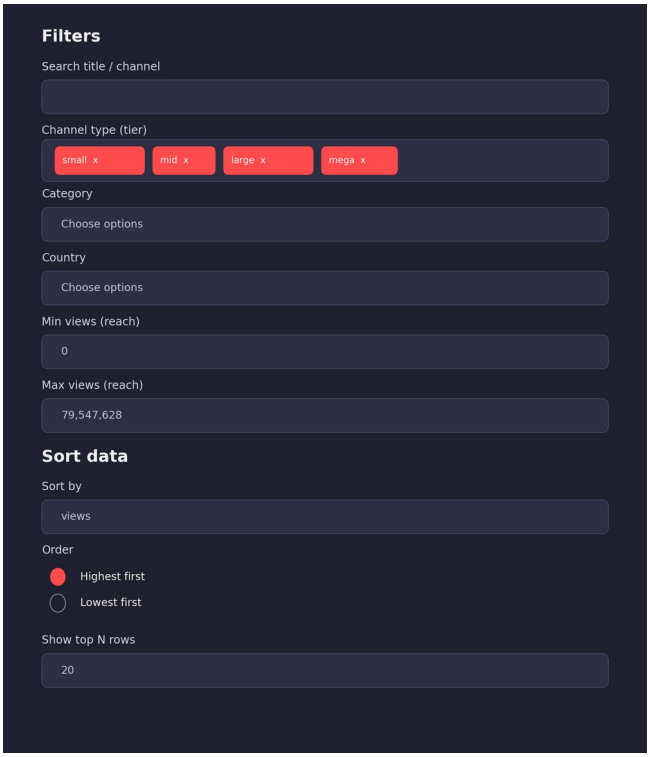


Figure 8.5: Filter and sort controls (tier, category, country, view range)

8.4 Analysis Outputs

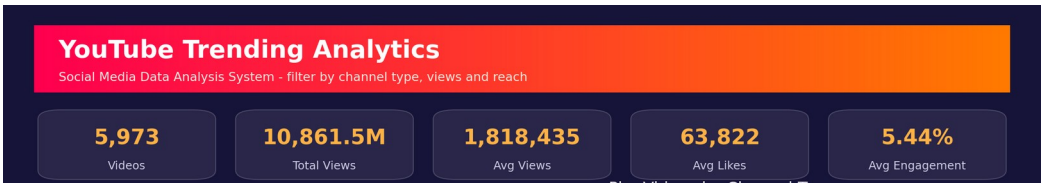


Figure 8.6: KPI summary cards

Top 20 videos sorted by views

video_id	title	tier	category	ctry	views	likes	cmts	eng. %
VID105318	New Trailer #433	mega	Music	IN	79,547,628	2,007,510	454,459	3.09
VID101579	Funny Trailer #602	mega	Gaming	IN	76,074,876	2,313,339	269,812	3.40
VID101908	Best Challenge #931	mega	Music	US	67,741,160	1,590,396	256,893	2.73
VID102514	Top Highlights #560	mega	Gaming	US	61,919,307	1,717,850	329,372	3.31
VID105087	Epic Mix #202	mega	News & Politics	GB	58,640,853	1,436,271	3,318	2.45
VID103727	Best Tutorial #796	mega	Music	MX	56,502,529	1,308,542	353,278	2.94
VID105762	Best Mix #877	mega	Gaming	GB	55,745,751	1,703,261	278,634	3.56
VID103913	Epic Highlights #5	mega	Education	CA	53,143,152	1,605,140	306,781	3.60
VID103941	Daily Review #33	mega	Science & Tech	IN	49,502,055	16,649	309,706	0.66

Figure 8.7: Sorted top-20 table of videos



Figure 8.8: Insights output

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Social Media Data Analysis System analyzed large-scale YouTube trending data with a clear methodology: collection, cleaning, exploratory analysis, visualization and insight generation. Category and country significantly influence video performance, and the findings help creators and marketers make informed, data-backed decisions.

9.2 Limitations

- The analysis depends on the quality and coverage of one fixed dataset.
- The project is descriptive and does not predict future trends.
- It does not use comment text, video content or thumbnails.
- It does not show causal relationships between category, country and views.
- Median filling of missing counts is an approximation.

9.3 Future Enhancements

- Comment sentiment analysis.
- Real-time trend prediction.
- Extension to Instagram, TikTok and other platforms.
- A hosted interactive dashboard with Streamlit or Power BI.
- Automatic data collection through the YouTube API.

CHAPTER 10
REFERENCES

- Python Documentation – Python Programming Language and Standard Library.
- Pandas Documentation – Python Data Analysis Library.
- NumPy Documentation – Numerical Computing with Python.
- Matplotlib Documentation – Python Data Visualization Library.
- Seaborn Documentation – Statistical Data Visualization.
- Google Colab Documentation – Cloud-based Python Environment.
- openpyxl Documentation – Reading and Writing Excel Files.
- Project dataset – youtube_large_dataset.xlsx, YouTube trending videos.
- Social Media Data Analysis System – DAE Project Presentation (Review-2) and project proposal presentation.

CHAPTER 11
APPENDIX – SOURCE CODE

Copy each cell below into its own Google Colab code cell and run them in order.

Cell 1: Imports and settings

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", 30)
pd.set_option("display.float_format", lambda x: f"{x:,.2f}")

OUT = Path("output")
OUT.mkdir(exist_ok=True)
```

Cell 2: Upload and load the dataset

```
FILE_NAME = "youtube_large_dataset.xlsx"

try:
    from google.colab import files          # works only inside Google Colab
    if not Path(FILE_NAME).exists():
        print("Choose youtube_large_dataset.xlsx from your computer ...")
        uploaded = files.upload()
        FILE_NAME = list(uploaded.keys())[0]
except ImportError:
    pass                                    # outside Colab: file must be in this
    folder

raw = pd.read_excel(FILE_NAME, sheet_name="Data")
print("Raw shape:", raw.shape)
display(raw.head()) if "display" in globals() else print(raw.head())
```

Cell 3: Initial exploration

```
print(raw.info())
print("\nMissing values per column:\n", raw.isna().sum())
print("\nDuplicate rows:", raw.duplicated().sum())
print("\nUnique values:\n", raw.nunique())
```

Cell 4: Cleaning

```
NUM_COLS  = ["views", "likes", "dislikes", "comment_count", "duration",
    "days_to_trend"]
DATE_COLS = ["publish_date", "trending_date"]

def clean(df):
    df = df.copy()

    # Step 1: tidy column names, drop completely empty columns
    df.columns = (df.columns.str.strip().str.lower()
```

```
                .str.replace(r"[^0-9a-z]+", "_", regex=True).str.strip("_"))
    df = df.dropna(axis=1, how="all")

    # check that the required columns exist (clear message instead of a KeyError)
    required = ["video_id", "title", "channel_title", "channel_tier", "category",
                "country", "views", "likes", "comment_count"] + DATE_COLS
    missing = [c for c in required if c not in df.columns]
    if missing:
        raise KeyError(f"Missing columns: {missing}. Columns found:
    {list(df.columns)}")
    num_cols = [c for c in NUM_COLS if c in df.columns]

    # Step 2: remove duplicate rows and duplicate videos
    df = df.drop_duplicates().drop_duplicates(subset="video_id")

    # Step 3: numbers numeric, dates real dates, negatives invalid
    for c in num_cols:
        df[c] = pd.to_numeric(df[c], errors="coerce")
        df.loc[df[c] < 0, c] = np.nan
    for c in DATE_COLS:
        df[c] = pd.to_datetime(df[c], errors="coerce")

    # text columns: remove extra spaces
    for c in ["title", "channel_title", "channel_tier", "category", "country"]:
        df[c] = df[c].astype(str).str.strip()

    # Step 4: drop rows with no date or no views, then fill other gaps with the
    median
    df = df.dropna(subset=DATE_COLS + ["views"])
    for c in num_cols:
        df[c] = df[c].fillna(df[c].median())

    # Step 5: remove zero-view videos
    df = df[df["views"] > 0]

    # Step 6: logical fixes
    for c in ["likes", "dislikes", "comment_count"]:
        if c in df.columns:
            df[c] = df[c].clip(upper=df["views"])          # cannot exceed views
    df = df[df["trending_date"] >= df["publish_date"]]     # cannot trend before
    publishing
    return df.reset_index(drop=True)

df = clean(raw)
print(f"Rows: {len(raw):,} -> {len(df):,} after cleaning")
print("Missing values left:", int(df.isna().sum().sum()))
```

Cell 5: Transformation and saving

```
def transform(df):
    df = df.copy()
    df["engagement_rate"] = ((df["likes"] + df["comment_count"]) / df["views"] *
    100).round(2)
    df["like_rate"]       = (df["likes"] / df["views"] * 100).round(2)
    df["log10_views"]     = np.log10(df["views"]).round(3)
    df["trend_month"]     =
    df["trending_date"].dt.to_period("M").dt.to_timestamp()
    return df

df = transform(df)
df.to_csv(OUT / "cleaned_youtube_data.csv", index=False, encoding="utf-8-sig")
df.to_excel(OUT / "cleaned_youtube_data.xlsx", index=False)
print("Saved cleaned files in:", OUT.resolve())
print(df[["views", "likes", "engagement_rate", "like_rate", "log10_views",
    "trend_month"]].head())
```

Cell 6: Descriptive statistics

```
print("Videos          :", f"{len(df):,}")
print("Total views     :", f"{df.views.sum()/1e9:,.2f} B")
print("Average views   :", f"{df.views.mean():,.0f}")
print("Median views    :", f"{df.views.median():,.0f}")
print("Average likes   :", f"{df.likes.mean():,.0f}")
print("Avg engagement  :", f"{df.engagement_rate.mean():.2f} %")
display(df[["views", "likes", "comment_count", "engagement_rate"]].describe()) if
    "display" in globals() else None
```

Cell 7: Charts

```
# 7.1 Bar - total views by category
cat_views = df.groupby("category")["views"].sum().sort_values()
ax = (cat_views / 1e6).plot(kind="barh", color="#1F4E79", figsize=(8, 5))
ax.set(title="Total Views by Category", xlabel="Total views (millions)",
    ylabel="")
plt.tight_layout(); plt.savefig(OUT / "bar_category_views.png", dpi=150);
    plt.show()

# 7.2 Pie / donut - videos by channel tier
tier_count = df["channel_tier"].value_counts()
plt.figure(figsize=(6, 6))
plt.pie(tier_count, labels=tier_count.index, autopct="%1.1f%%", startangle=90,
        wedgeprops=dict(width=0.55))
plt.title("Share of Videos by Channel Tier")
plt.savefig(OUT / "pie_channel_tier.png", dpi=150); plt.show()

# 7.3 Line - total views by trending month
month_views = df.groupby("trend_month")["views"].sum().sort_index()
ax = (month_views / 1e6).plot(kind="line", marker="o", figsize=(9, 4.5))
ax.set(title="Total Views by Trending Month", xlabel="Month", ylabel="Total views
    (millions)")
plt.tight_layout(); plt.savefig(OUT / "line_monthly_views.png", dpi=150);
    plt.show()

# 7.4 Histogram - views on log scale
plt.figure(figsize=(8, 4.5))
plt.hist(df["log10_views"], bins=30, edgecolor="black")
plt.title("Distribution of Views (log10 scale)")
```

```
plt.xlabel("log10(views)"); plt.ylabel("Number of videos")
plt.savefig(OUT / "hist_log_views.png", dpi=150); plt.show()

# 7.5 Box plot - engagement rate by channel tier
order = df.groupby("channel_tier")["engagement_rate"].median().sort_values().index
plt.figure(figsize=(8, 4.5))
sns.boxplot(data=df, x="channel_tier", y="engagement_rate", order=order,
    showfliers=False)
plt.title("Engagement Rate by Channel Tier")
plt.xlabel("Channel tier"); plt.ylabel("Engagement rate (%)")
plt.savefig(OUT / "box_engagement_tier.png", dpi=150); plt.show()

# 7.6 Scatter - views vs likes (log-log)
plt.figure(figsize=(7, 5))
d = df[df["likes"] > 0]
plt.scatter(d["views"], d["likes"], alpha=0.25, s=10)
plt.xscale("log"); plt.yscale("log")
plt.title("Views vs Likes (log-log)"); plt.xlabel("Views"); plt.ylabel("Likes")
plt.savefig(OUT / "scatter_views_likes.png", dpi=150); plt.show()

# 7.7 Country-wise average engagement
ctry_eng = df.groupby("country")["engagement_rate"].mean().sort_values()
ax = ctry_eng.plot(kind="barh", color="#7F6000", figsize=(7, 4.5))
ax.set(title="Average Engagement Rate by Country", xlabel="Engagement rate (%)",
    ylabel="")
plt.tight_layout(); plt.savefig(OUT / "bar_country_engagement.png", dpi=150);
    plt.show()

# 7.8 Correlation heatmap
corr_cols = [c for c in ["views", "likes", "dislikes", "comment_count",
    "duration",
                         "engagement_rate"] if c in df.columns]
corr = df[corr_cols].corr().round(2)
plt.figure(figsize=(7, 5.5))
sns.heatmap(corr, annot=True, cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Correlation Heatmap")
plt.tight_layout(); plt.savefig(OUT / "heatmap_corr.png", dpi=150); plt.show()

# 7.9 Category x Country heatmap of average views
piv = df.pivot_table(index="category", columns="country", values="views",
    aggfunc="mean") / 1e3
plt.figure(figsize=(10, 6))
sns.heatmap(piv, annot=True, fmt=".0f", cmap="YlGnBu")
plt.title("Average Views (thousands): Category x Country")
plt.tight_layout(); plt.savefig(OUT / "heatmap_cat_country.png", dpi=150);
    plt.show()
```

Cell 8: Filters, sorting and top-N table

```
def dashboard(tiers=None, cats=None, countries=None, min_views=0, max_views=None,
              search="", sort_col="views", ascending=False, top_n=10):
    f = df.copy()
    if tiers:     f = f[f["channel_tier"].isin(tiers)]
    if cats:      f = f[f["category"].isin(cats)]
    if countries: f = f[f["country"].isin(countries)]
    max_views = df["views"].max() if max_views is None else max_views
    f = f[f["views"].between(min_views, max_views)]
    if search:    # plain-text search in title or channel name (no regex errors,
    NaN safe)
        f = f[f["title"].str.contains(search, case=False, regex=False, na=False)
```

```
            | f["channel_title"].str.contains(search, case=False, regex=False,
    na=False)]

    if f.empty:
        print("No videos match these filters.")
        return f

    kpis = {"Videos": f"{len(f):,}",
            "Total Views": f"{f.views.sum()/1e6:,.1f}M",
            "Avg Views": f"{f.views.mean():,.0f}",
            "Avg Likes": f"{f.likes.mean():,.0f}",
            "Avg Engagement": f"{f.engagement_rate.mean():.2f}%"}
    print(" | ".join(f"{k}: {v}" for k, v in kpis.items()))

    cols = ["title", "channel_title", "channel_tier", "category", "country",
            "views", "likes", "comment_count", "engagement_rate"]
    table = f.sort_values(sort_col, ascending=ascending).head(int(top_n))[cols]
    display(table) if "display" in globals() else print(table)
    return f

f = dashboard(top_n=10)                                    # all data, top 10 by
    views
_ = dashboard(tiers=["Mega", "Large"], cats=["Music"],     # example filtered view
              min_views=100_000, sort_col="engagement_rate", top_n=5)
```

Cell 9: Insights

```
def insights(f):
    top_cat     = f.groupby("category")["views"].sum().idxmax()              #
    FIX: was bar.iloc[-1]
    best_tier   = f.groupby("channel_tier")["engagement_rate"].mean().idxmax()
    top_country = f.groupby("country")["views"].sum().idxmax()
    print("Top category (most total views)      :", top_cat)
    print("Best-engaging channel tier            :", best_tier)
    print("Top country (most total views)        :", top_country)

insights(f)
```

Cell 10 (optional): Interactive filters and download

```
try:
    import ipywidgets as w
    w.interact(
        dashboard,
        tiers=w.SelectMultiple(options=sorted(df.channel_tier.unique()),
    description="Tier"),
        cats=w.SelectMultiple(options=sorted(df.category.unique()),
    description="Category"),
        countries=w.SelectMultiple(options=sorted(df.country.unique()),
    description="Country"),
        min_views=w.IntText(value=0, description="Min views"),
        max_views=w.fixed(None),
        search=w.Text(value="", description="Search"),
        sort_col=w.Dropdown(options=["views", "likes", "comment_count",
    "engagement_rate"], description="Sort by"),
        ascending=w.Checkbox(value=False, description="Ascending"),
        top_n=w.IntSlider(value=10, min=5, max=50, step=5, description="Top N"),
    )
except ImportError:
    print("ipywidgets not installed - use dashboard(...) calls above instead.")
```

```
try:
    from google.colab import files
    files.download(str(OUT / "cleaned_youtube_data.csv"))
except ImportError:
    pass
```

***** END OF REPORT *****